Workloads and Scheduling for CKA

Core Workload Primitives

Workloads and Scheduling in the CKA Exam

Hello, and welcome to the course Workloads and Scheduling for CKA. My name is Antonio Piedra, and I'll be your guide throughout this course. If you're preparing for the CKA exam, you already know that this domain accounts for 15% of the total score, but beyond the points, this is probably the most real-world part of the exam. While other domains focus on keeping the cluster itself healthy, this section is about keeping the applications healthy. This domain tests your knowledge on application lifecycle management. How do we deploy an app? How do we update it without dropping users? If a pod crashes at 3 a.m., how does Kubernetes fix it automatically? This section of the exam also focuses on where the containers are deployed. In a large cluster, not every node is equal, so selecting the correct node for each pod is critical. To be more specific, this domain covers application deployment, rolling updates and rollbacks, how to configure applications with secrets and ConfigMaps, how workloads can be automatically scaled, for example, when the number of users increase, learning about the self-healing capabilities that Kubernetes has and understanding how pod admission and scheduling works.

Choosing the Right Workload Controller

In Kubernetes, we rarely create a standalone pod. Why? Because a standalone pod has no one looking after it. If it dies, it stays dead. Instead, we use controllers. Think of a controller as a manager with a desired state. Their only job is to watch the cluster and make sure that the actual number of pods matches the number you ask for. But which controller should you pick? First, we have deployment. This is the CKA default and the most common objects you'll use. Deployments are designed for stateless applications like a web server, APIs, or anything where one pod is exactly like the next one. It manages the running update process, allowing you to update your code with zero downtime. Next is the StatefulSet. As the name suggests, this is for stateful applications like databases. Unlike deployment, pods in a StatefulSet have a stable, unique identity. If database0 restarts, it comes back as database0, keeping its name and its persistent storage. Then there's the DaemonSet. Think of a DaemonSet as a per node requirement. It ensures that one copy of a pod is running on every node in your cluster. If you add a new node to the cluster tomorrow, the DaemonSet controller will automatically spin up a pod on it. This is perfect for background tasks like log collection or network monitoring. Finally, we have jobs and

CronJobs. Everything that we have mentioned so far is meant to run forever, but what if you just need to process a batch of images or run a database backup? A job runs a task until it completed successfully, and then it stops the pods. A CronJob does exactly the same thing, but on a recurring schedule, like every Sunday at midnight. Let's look at how this compares side by side as this is a common way the exam tests your architectural knowledge. Deployments and DaemonSets are long-running processes. They are meant to stay up. Jobs, however, are temporary. They start, they do the work, and then they exit. The scaling mechanism also differs. While you scale a deployment manually or via an autoscaler, for example, a web server when the number of users increases, a DaemonSet scales automatically based on your cluster size, one per node, very useful, for example, to collect logs on each of the nodes. Jobs, however, scale based on completions. Essentially, this is how many times you want the task to succeed. Jobs are very useful when you need to back up or migrate data. Understanding when to choose the right controller is critical for the CKA exam, so make sure that you have this clear.

Demo: Managing Long-running Workloads

Let's start with the first demo. Remember that you can download these files from the Resources tab. In this first demo, we will learn how to quickly deploy a DaemonSet. We will watch Kubernetes self-healing in action when a pod from the DaemonSet is deleted, and also we will learn how labels are used to identify the children pods of a DaemonSet. Before we get started, let's look at the cluster setup. I have already done this setup, but essentially, you will need to download and install Docker and also install Minikube. Then, you will need to start Minikube with three nodes and create a namespace for each of the modules. For each of the modules, you will also create a context so we can quickly switch between namespaces. And to switch between the context, you will use the kubectl config use-context and the context name command. In each of the questions in the CKA exam, you will be provided with a kubectl config use-context command. Make sure that you execute that command so you don't end up doing changes in other namespaces. I also recommend setting an alias for kubectl. This way, instead of typing kubectl each time, you can just type k. This is much faster and avoid typos. I also recommend using kubectl to generate boiler template YAML for you. You can create an environment variable called do that stores the –dry-run=client and the output YAML parameter. This saves time as you don't need to generate the YAML definition yourself and also avoid typos. Now that you are done with the setup, let's go back to the demo. The first thing that we're going to do is switch to the correct context. Then, we will create a deployment definition. And note we have used here k as the alias for kubectl and also the variable do, which is stored in the –dry-run=client and also output YAML. The output is saved in a file called ds.yaml. Let's now convert the deployment definition into a DaemonSet. For that, we will open the file with vim. And then we

will remove the replicas field and also the strategy field and replace the kind with DaemonSet. We will save the file And then we are going to use the kubectl apply -f command to create the DaemonSet. As I have three nodes in my environment, I should have three pods, one in each of the nodes. Let's execute the kubectl get pods command. Using the -o wide parameter gives you additional information about the pod, like the node where it has been scheduled and also the pod IP. We can see that we have a pod in each of the nodes. Let's now delete one of the pods, for example, the one that is running on the third node. For that, we will use the kubectl delete po and the name of the pod. Then we will run again the kubectl get pods command. And we see that a new pod has been created 11 seconds ago. This means that the DaemonSet controller was able to identify that a pod was deleted and automatically started a new one in the correct node. Remember, a DaemonSet ensures that a pod is running in each of the nodes. Let's have a look again at the DaemonSet definition. We have the selector field, which is matching labels. In this case, it's just matching a single label called up with the value Logger. What will happen if we edit one of these pods and modify the value of the label? Let's do that with the pod that was started on the third note. For that, we will use the kubectl edit po and the name of the pod. I will modify the value of the app label to be logger-x. After saving the file, I will check again the running pods. And we see that the DaemonSet has started a new pod. So essentially, the DaemonSet is using the app logger label to identify its children. As we modified the label in one of the pods, it thought that one of the pods disappeared, so it created a new one. Let's now delete the DaemonSet with the kubectl delete ds command. Now we will check again the running pods. And we see that the DaemonSet and the three pods from the DaemonSet were deleted, but the one that we modified was not deleted. This is what we call an orphan pod. It doesn't have a parent that is taking care after it. So remember the importance of labels and selectors. Do not modify them after creation unless it's really required. Now, to delete the orphan pod, we will need to run the kubectl delete pod and the pod name command.

Demo: Managing Batch Workloads

Let's now work with batch workload. In this demo, we will create a job and understand its success criteria. We will configure job completions and parallelism, and also we will schedule the task to run as a CronJob. The first thing that we're going to do, as you should do in the exam, is switching the context. After that, we will create a job definition with the kubectl create job command. This command created the job.yaml file, which contains a job definition. The name of the job is calculation, and it runs a single container with the busybox image which sleeps for 5 seconds. We will run the kubectl apply -f command to create the job. Then, we will execute the kubectl get pods command. And we can see that the status is currently running. If we execute it again, we can see that it has completed. If we run the kubectl get job command,

we can see the job status. It took 9 seconds, and we can see that the status is Complete. Now we are going to edit the job definition to configure completions and also parallelism. We will set completions to 6, and this is the number of required successes. So essentially for the job to be marked as completed, the pod needs to run six times successfully. We will also configure parallelism. We will set it to 2, and this means that a maximum of two pods can be run at the same time. We will save the file, and then we will run the kubectl apply command to create the job again. Now let's watch as the pods progress. And then we can see that four of them already completed, and now it's starting to run the last two. Let's give it a second, and we see that the six pods already completed in batches of two. Now let's delete the job. And we are going to schedule a job to run every minute via a CronJob. We can do this by executing the kubectl create cronjob command. In this case, the name of the CronJob is backup, it's using the busybox image, and in this case, it's using the echo command. Now we are going to use the kubectl get job command. If we wait for 1 minute, we will see a new job being created. And we can see it here, the second job has been created after 1 minute. Let's say that you have a job that performs a database backup. If you want to perform the backup every single night, you can use a CronJob definition to run that job every day. Let's now delete the CronJob using the kubectl delete cronjob backup command.

Application Lifecycle Management

Deployment Strategies

We're going to talk about the most common task you will face as an administrator, the update. Let's say that you have version 1 of your application running in production. Your developers just handed you version 2. What are your options to handle the updates? In Kubernetes, we have two primary strategies. First, there is the recreate strategy. Think of it as the nuclear option. Kubernetes kills all version 1 of your pods at the same time, waits for them to terminate, and only then it starts the version 2 pod. It's simple, and it ensures you never have two different versions of your application running at the same time, but it results in a guaranteed downtime. In the CKA exam, you'll rarely use this unless explicitly asked. The second and far most common method is the rolling update. This is the default behavior in Kubernetes. It replaces all pods with the new ones, one by one, ensuring that at least some version of your application is always available to handle traffic. To master the running updates for the exam, you need to understand two key throttle settings. MaxSurge defines how many extra pods you are willing to run above your desired counts during the update. If you have four pods and a maxSurge of 25%, Kubernetes will spin a new version 2 pod before killing any version 1 pod. MaxUnavailable is the opposite. It defines how many pods can be offline at any given time. By tweaking these two numbers, you

can control exactly how fast or how safe your rollout is.

Demo: Performing Rolling Updates

Let's see running updates in action. In this demo, we will trigger an update by changing a container image. We will monitor the update in real time, and we will verify that the new version is active. The first thing that we are going to do is switch the context to the module2-context. Then, we will create a deployment called web-server that runs three replicas of the Nginx image. In this case, we are using version 1.16. Then, we will execute the kubectl get pods command, and we see that we have three pods running. Now let's update the Nginx version to 1.17. For this, we will use the kubectl set image command. Then with the kubectl rollout status command, we can monitor the rollout, and we can see that the rollout has been successfully completed. If we also use the kubectl describe command, we can see that the new image version is 1.17. Let's update the Nginx version again. We will update it to 1.18. After that, we will watch with the kubectl get pod command. And it happened very fast, but we could see that all pods are deleted one by one once the new pods are ready. We can also check the rolling the configuration by using the kubectl edit deploy command. We can see maxSurge set to 25%, so this means that 25% of the pods can be created over the desired number of replicas, and also the maxUnavailable is 25. So we are only allowing 25% of the pods to be unavailable during updates. If you want a faster rollout, then you can set these values to 50%, for example.

Rollback and Revision History

In a production environment, updates are the most dangerous moments in an application lifecycle. To manage this risk, Kubernetes provides a built-in time machine called revision history. Every time that you modify the pod templates of a deployment, whether you are changing the image version, the labels, or the environment variables, Kubernetes creates a new revision. In this example, Revision 1 included version 0.0.1 of the image. Then, the version of the image was updated to 0.0.2 to include an additional feature of the application. Then in version 3, the team tried to fix a bug, but they introduced a change that prevented the pod from starting. In this example, for each of the versions, Kubernetes created a revision of the deployment. But remember that the deployment does not actually create the pod. The deployment just creates a ReplicaSet, and the ReplicaSet controls the pod. So every time that you modify something in the deployment manifest, it just creates a new ReplicaSet. The rollout history is actually just a collection of old ReplicaSets kept in the cluster. By default, Kubernetes stores 10 revisions. If you perform an eleventh update, the oldest ReplicaSet, in this case, revision 1, is deleted to make room for the new one. You can also define how many old revisions to keep using

the revisionHistoryLimit field in your deployment manifest. In the exam, you will need to know how to inspect this history using the rollout history command. This allows you to see the list of safe points that you can return to. You can even roll back a specific revision of two or three days ago if needed.

Demo: Performing Rollbacks

How do we handle rollbacks in a deployment? In this demo, we are going to view the history of changes in a deployment. We will simulate a failed deployment by using a broken image, and finally, we will undo the change to return to a stable state. The first thing that we are going to do is set up the correct context. Then, we will check the rollout history of the deployment by using the kubectl rollout history, deployment, and then the name of the deployment. We can see that the web-server deployment has three revisions. Remember that the first version of the deployment was using Nginx version 1.16, the second revision was using version 1.17, and the third one was using 1.18. Now we are going to update the deployment again to set an image that doesn't exist. In this case, we are going to use the pluralsight tag. If we now check the pod status with the kubectl get pod command. We can see that a new pod has been created, but it has failed with an ImagePullBackOff error. So the new deployment version is broken. We need to roll back as soon as possible. To do this, we can execute the kubectl rollout undo command. We can verify the status of the rollback with the kubectl rollout status, and we can see that the deployment has been successfully rolled out. If we execute again the kubectl get pod command, we see that the pod from the broken version has also been deleted. Lastly, if we check the rollout history, we see that we have revision 1, 2, 4, and 5. Revision 3 was the stable one and 4 was the broken one. So when we did the rollout undo, version 3 became 5 to be the active one.

Configuration and Decoupling

Decoupling Configuration

Let's look at why decoupling configuration is so important. Have you heard about the twelve-factor app before? It's a methodology for building modern cloud-native Software as a Service applications that are portable, scalable, and resilient. It was created by Heroku engineers and leads down the 12 best practices to minimize setup time for new developers, maximize portability between environments, and enable continuous deployment. All the 12 principles are worth having in mind, so I would recommend having a look at them. For now, we are going to focus on factor 3, store configuration in the environment, not in code. How can we do that? Let me try to explain it to you with a simple use

case. Let's say that you have an application that you need to deploy to your three environments dev, test, and production. Based on your environment, the application connects to a different web server, dev.pluralsight.com, test.pluralsight.com, and prod.pluralsight.com. So to support this, you have a pipeline that generates three images, one for each of the environments. The image is pretty much the same. The only thing that changes across all of them is the domain. What about if you move the configuration to the environment itself to your Kubernetes cluster? Then, you just need to have a single image which uses the base variable and fetches the domain from whatever environments you are running on. Decoupling just made your build process much faster. You just need to generate a single image instead of three. Also, this means that you only need to store an image instead of three in your container image registry. In the previous example, the configuration getting decoupled wasn't too sensitive. It was just the domain name. But what about if the application needs credentials to authenticate against the web server? Where do you store that password? I hope not hardcoded in your application code. This is another example where decoupling works. You should store your credentials somewhere else and inject them in the container when it starts. This way, even if someone has access to your code, they won't see the password. How can Kubernetes help with decoupling? There are two Kubernete objects that are extremely useful, ConfigMaps for non-sensitive information like URLs or domains like in our example, and secrets, intended for sensitive data like password tokens or SSH keys. So you can define these two objects. How can the container access them? There are two methods in Kubernetes. Inject them as environment variables so when your application starts it reads them from the environment, or the second injection method is via volumes, essentially creating the configuration or secret into the container file system. For small things like password, using environment variables works very well. For bigger configurations like a web server configuration, it's better to use volumes. One thing to have in mind is that volume supports hot reloading. Environment variables do not, so you will need to restart the pod to update the values.

Demo: ConfigMaps and Environment Variables

In this demo, we are going to look at ConfigMaps and environment variables. We will create a ConfigMap using kubectl that will store our environment details, for example, dev, test, or production. We will inject that ConfigMap into an Nginx pod as an environment variable, and finally, we will validate that the injection worked by connecting to the Nginx pod. As we have done previously for this demo, we will be using two shortcuts. The first one is using an alias for kubectl, in this case, k. So that way, we reduce typing and we are faster in the exam. The other thing that we are going to do is create a shorthand variable to quickly generate boiler template YAMLs without the need of typing the flags. Now, let's start with the demo. The first thing that we are going to do is switching the context to the

module3-context. You will need to run this command in the CKA exam. Then, we will create a ConfigMap that will store the environment name, pluralsight-dev, and also the APP_MODE, in this case, TEST. The name of the ConfigMap is going to be web-env-config. We will verify that the ConfigMap has been created by using the kubectl get cm for ConfigMap and the name of the ConfigMap. As we can see, the ConfigMap was created 10 seconds ago, and it contains two items of data. Now, we will create a YAML definition of a pod running the Nginx image. Note that we are using the shorthand that we created before. Now, we are going to edit the definition to add the ConfigMap as an environment variable. For this, I will open the pod definition with vim, and I will add the environment from within the first container. The indentation is very important in YAML files, so make sure that you fix it. Then, I will save the file. And we are going to start the pod by using the kubectl apply -f command. The pod has been created, so now we can execute the kubectl get pods to see the running pod. And as we can see, the pod was started 6 seconds ago and the status is running. Now we are going to check the environment variables within the pod. If we make the screen bigger, we can see the APP_MODE=TEST and also pluralsight-dev set up. So this means that the data from the ConfigMap is being injected as environment variable into the container as expected. You could, for example, use these environment variables in your code to decide what environment you are on and also the mode that your applications operates on. That will be all for this demo, so let's execute the kubectl delete pod to remove the pod, and also the kubectl delete cm for ConfigMap to delete the ConfigMap.

Demo: Hot Reloading Configuration

Let's now look at hot reloading configuration, something that you can only do with volumes. In this demo, we will expose the Nginx landing page. We will then configure the index.html page to be stored in a ConfigMap, and after that, we will do changes in the ConfigMap and verify that they get reflected on the fly, basically hot reloading. As mentioned in other demos, for you to be faster in the exam, I recommend setting up an alias for kubectl and setting up a shorthand variable to quickly generate boiler template YAMLs for you. The first thing that we are going to do is switching to the correct context. After that, we will create a YAML definition of a pod running the Nginx image. We will use the shorthand variable that we defined before. We will then create the pod with the kubectl apply command. And we can see that the pod has been created, so now we will verify that the pod is running with the kubectl get pod command. We can see that the pod was started 13 seconds ago and the status is set to Running. We are now going to expose the Nginx pod as a NodePort service. In this case, as Nginx listens in port 80, that will be our target port. The service has been created, so we will verify by using the kubectl gets svc for service command. As we are using Minikube, we need to also execute the minikube service and the name of the service, expose the service. After doing this, the browser will open

automatically. Let's now open another terminal. We will check the index.html page that we are going to use in our ConfigMap. As we can see, it's a very simple HTML page that contains a pink background. Let's go back to the instructions, and now we are going to create a ConfigMap called web-html using the file as a source. The ConfigMap has been created, and we can see it by using the kubectl get cm for ConfigMap. We have two ConfigMaps, one with the kube-root-ca that was created automatically, and also the ConfigMap that we just created. Now we are going to update our pod definition to override the index.html. We will need to do two things. The first thing that we are going to do is define the volume. We will do this within the spec field. And also within the container, we will need to define a volumeMount that references the volume that we just added. The name of the volume is html-dir, and the mount path is usr/share/nginx/html. For hot reloading to work, it's very important that you reference a whole directory and not a file. Now we're going to go back, and we are going to delete the previous pod with the kubectl delete pod and the name of the pod. And we are going to execute the kubectl apply -f to create the pod. The new pod has been created, and now if we go back to the browser and refresh the page, we will see the new page with the pink background. So the Nginx container was able to obtain the index.html page from the ConfigMap. Now let's see how hot reloading works. We are going to use the kubectl edit cm for editing the ConfigMap, and we are going to do a very simple change. We are going to change the background color to be black. We will save the changes, and we will execute the kubectl get cm to verify that the changes have been saved as expected. And we can see that the background color is set to black. We are also going to check inside the pod. For this, we are going to execute the kubectl exec, the pod name, dash dash, and the command that we are going to execute. In this case, the command that we are going to execute is cat and the full path to the HTML page. If we make the terminal window a little bit bigger, we can see that the background color has also changed to black within the container, so hot reloading seems to be working. Let's now go back to the browser and refresh the page. Great, the background of the page is now black, so hot reloading is working as expected. To wrap up this demo, we will stop the minikube expose command. We will delete the pod with the kubectl delete pod command. We will also delete the service. And finally, we will delete the ConfigMap.

Managing Secrets

A secret is a Kubernetes resource intended to store a small amount of sensitive data like password, tokens, or API keys. Such information might otherwise be put into a pod specification or a container image. Using a secret means that you don't need to include your confidential data into your application code. Because secrets can be created independently of the pods that use them, there is less risk of a secret and its data being exposed during the workflow of creating, viewing, or

editing pods. Essentially, secrets are similar to ConfMaps, but specifically intended to hold confidential data. When creating a secret, you can specify its type. You can always use the opaque type. It's the default one if you don't specify any, but Kubernetes provides a set of built-in types for common usage scenarios. Kubernetes validates the data stored in the secrets based on the configuration type. There are types of secrets to store service account tokens, dockercfg and docker/config.json to store image registry credentials, basic-auth to store credentials needed for basic authentication, ssh to store SSH privacy keys, tls to store certificates, and the Bootstrap token used during the node Bootstrap process. If using these built-in types, you must adhere to the governance validation. For example, if using the TLS type, you will need to specify the tls.key and the tls.crt keys. You can also define your own secret type. In doing so, you should follow the convention and structure for the secret type, which is your domain name, for example, cka.pluralsight.com/ and the name of the secret type. We have been saying that secrets are used to store confidential information like password or SSI keys, so secrets must be secured, right? The answer is that it depends. Secrets are Base64 encoded, which obscures them, but does not provide any useful level of confidentiality. Also, by default, secrets are stored and encrypted in the etcd. This means that anyone with API or etcd access can retrieve or modify a secret. Additionally, anyone who is authorized to create a pod in a namespace can use that access to read any secret in that namespace. This includes indirect access, such as the ability to create a deployment. So what can you do to increase the security of your confidential information? By default, secret objects are stored and encrypted in the etcd. You should configure encryption of your secret data in etcd. Use the principle of least privilege. If using role-based access control, RBAC, restrict access to verbs like get, watch, or list to the secret resource; otherwise, anyone with access to the cluster will be able to retrieve your sensitive data. If using a pod with multiple containers and only one of those containers needs the secret, ensure to mount the volume or environment variable only in that single container rather than doing it at pod level. And lastly, you can also consider using third-party secret store providers to store your secrets outside of the Kubernetes cluster. HashiCorp Vault is an example of a third-party secret provider.

Demo: Mounting Secrets into a Pod

In this demo, we are going to cover how to mount a secret into a pod. We will create a secret using kubectl to store a password. We will inject the secret into an Nginx pod as an environment variable, and finally, we will validate that the injection worked by connecting to the Nginx pod. As we have done in other demos, we will set up an alias for kubectl, and also we will set up a shorthand variable to quickly generate boiler template YAML. We will start by switching the context to module3, and we will create a secret to store the password. For this, we will use the kubectl create secret generic, then the name of the secret, db-

pass, in this case, –from-literal, and we will specify the name of the key and also the value. In this case, the key is DB_PWD and the value will be K8sRocks!. We will then verify that the secret has been created with the kubectl get secret command. We can see that the secret was created 9 seconds ago. We can also use the -o yaml to output the secret in YAML format. And we can see that the secret is Base64 encoded. Let's now decode the secret using echo, then the secret value, pipe, and then base64 -d for decoding. And we are able to decode the secret, so make sure that you limit who can get, list, or modify your secret. Next, we are going to create a YAML definition of a pod running the Nginx image. We will then edit the pod definition to add the secret as an environment variable. Within the container, we'll specify the envFrom key, and we will define the secret reference and specify the name of the secret, in this case, db-pass. We will then create the pod with the kubectl apply -f command. And we will verify that the pod is running by using the kubectl get pods command. We can see that the pod was created 10 seconds ago and the status is Running. Let's now execute the printenv command within the pod. For this, we will use the kubectl exec, the name of the pod, dash dash, and the command to execute. We can see the DB_PWD environment variable with the password that we set up before. Similar to what we mentioned before for secret, do limit who can access your pod because they will be able to retrieve your secret. Now, as we have the password as an environment variable in the pod, if your application needs to connect to a database, it can pull the password from this environment variable. That will be all for this demo, so let's perform some clean up. The first thing is delete the pod by using the kubectl delete pod command. And then we will execute the kubectl delete secret and the name of the secret to remove the secret resource.

Pod Admission and Scheduling

Configuring Pod Admission

Before a pod even reaches a node, it must pass through the admission phase. This is where Kubernetes determines if the pod is fit for the cluster based on its resource requirement. The most critical part of a pod definition is the resources block. We define two things, requests and limits. Requests are the guaranteed resources. The scheduler uses this number to find a node that has enough spare capacity. If you request 500m of CPU, Kubernetes will only place that pod on a node that can truly provide them. Limits, on the other hand, are the hard ceiling. They prevent a pod from consuming too much resources on a node that could impact other pods running on the same node. If a container exceeds its CPU limits, Kubernetes simply throttles it, slowing it down. But if it exceeds its memory limits, the kernel will terminate the process, resulting in an Out of Memory Killed status. Based on how you define these numbers, Kubernetes assigns a Quality of Service class to your pod. This determines the pod's priority

during a resource crunch. Guaranteed is the highest priority. It is assigned when the request and the limits are identical for both CPU and memory. Burstable is the middle tier where requests are lower than the limits. And finally, BestEffort is the lowest priority, assigned when there are no resources defined. In a scenario where a node runs out of memory, BestEffort pods are the first ones to be evicted, while Guaranteed pods are protected until the very last resort.

Demo: Defining Resource Limits

In this demo, we are going to learn how to configure resource limits. We will define requests and limits for a pod, we will monitor the pod usage, and finally, we will simulate out of memory errors. As we have done in previous demos, we will set up an alias for kubectl and also export a shorthand variable to quickly generate boiler template YAML without manually typing all the flags. We will then switch to module4-context. We will execute the kubectl run command to generate a basic pod definition. The pod is going to be called monitor-pod, and it's going to run the /vish/stress image. It's an image used for stress testing. Then we will update the pod definition for the pod to consume one CPU. We will open the YAML, and within the container, we will add the arguments. We will then start the pod with the kubectl apply command, and we will verify that the pod is running with no issues with the kubectl get pods command. We see that the pod was starting 9 seconds ago and the status is set to Running. Let's now inspect the Quality of Service using the kubectl describe command. If we expand the terminal, we can see that the Quality of Service class is BestEffort. This is because we haven't defined any requests or limits for our pod. Let's monitor the resource usage with the kubectl top command. And we can see that the pod is using almost one full CPU. Let's now define requests and limits for the pod. We will open again the pod definition. And we are going to limit the CPU to 500m. Let's now delete the old pod with the kubectl delete pod command and recreate it again with the kubectl apply command. Let's verify that the pod is running with no issues and also inspect the Quality of Service class that Kubernetes has assigned. Now, the Quality of Service class is set to Burstable since the requests are lower than the limits. Let's now check the resource usage. As we can see, Kubernetes is already limiting the CPU usage. Let's go back to the pod definition and set up even tighter limits. For the memory, we will request only 5Mi and limit it to maximum 10Mi. Let's delete the pod and then create it back again. And let's check the pod status with the kubectl get pod command. If we execute the command several times, we can see that the container is restarting. We see that the pod is not running and it has a status of OOMKilled. This means that Kubernetes automatically killed the pod because it was consuming more memory than what we set up as a limit. Let's now clean up the pod with the kubectl delete pod command.

Pod Scheduling

The decision on where to run a pod runs on the Kubernetes scheduler. When you create a pod, the definition reaches the scheduler, which decides if the pod should run on Node 1, Node 2, or Node 3, based on the resource and limits you define. While the scheduler is highly efficient, there are many times you will need to override its decisions to meet a specific hardware or security requirements. For this, we have three options. Using nodeSelector is the simplest way to control placement. You assign a label to a node, for example, disk SSD, and then you add the very same label to your pod in the nodeSelector field. If the scheduler finds a node with the exact key value pair, the pod is placed there. If no nodes match, your pod remains in a pending state, so you need to be careful there. Another option is to use taints and tolerations. This is an exclusionary system. Instead of a pod choosing a node, the node uses attains to repel 13 pods. This is how we reserve nodes for specific workloads or keep application pods off the control plane. To bypass a taint, a pod must have a matching toleration. And the last and more powerful and flexible option is affinity and anti-affinity. Node affinity is an evolved version of the nodeSelector. It supports complex operators and, more importantly, soft rules known as preferred rules. This allows the scheduler to try and find a specific node, but still schedule the pod elsewhere if that node is full. Pod anti-affinity is used for high availability. It allows you to tell Kubernetes do not put two replicas of these pods on the same node. This ensures that if one of the nodes fails, your entire application doesn't go down with it. Mastering these three methods ensures that you can handle any scheduling scenario the CKA exam and the real world throws at you.

Demo: Node Selection

In this demo, we are going to use labels to instruct the Kubernetes scheduler where it should run the pod. We will add a label to a node. We will then create a pod with that label defined in the nodeSelector field, and also we will check what happens if we create a pod with a nodeSelector that doesn't match any of the nodes. Before we start the demo, we will set up the alias for kubectl and also the shorthand variable to quickly generate boiler template YAMLs. Then we will switch to module4-context, similar to what you will do in the exam, and then we are going to execute the kubectl get node command to list the number of nodes. As we can see, we have three nodes. The minikube node is the control-plane, and we have two worker nodes, minikube-m02 and minikube-m03. We will add a label to the node with the kubectl label node, the name of the node, and the label, in this case, disk=ssd. And we can see that the node has been successfully labeled. To verify, we will execute the kubectl get nodes and filter by label disk, and we can see that only node 2 has the label. Now we will create a pod definition with the kubectl run command. The pod name is going to be

selector-pod, and it's going to run an Nginx image. Now we are going to add the nodeSelector field to the pod definition. In this case, we are selecting nodes that have a disk label with the ssd value. We will create the pod with the kubectl apply command. And using the kubectl get pods -o wide, we will verify that the pod is running on the expected node. And here we have it. It's running on node number 2. But what will happen if we configure on the nodeSelector field a label that doesn't exist in any of the nodes? Let's try that out. Let's go back to the pod definition and update the label value to something that does not exist. Let's delete the pod with the kubectl delete pod command and create it again with kubectl apply -f and the name of the file. Let's check now the pod status with the kubectl get pods command. And we see that the pod status is set to Pending. If we verify the Kubernetes events with the kubectl gets events command, we see that the pod failed to be scheduled because no node matched the nodeSelector. So when you see nodeSelector, you need to be careful because your pod may end up not being scheduled at all. Let's now do some clean up with the kubectl delete pod command to delete the pod. Also remove the disk label from the node 2 with the kubectl label node, the name of the node, and then the name of the label followed by a dash to remove.

Demo: Taints and Tolerations

In this demo, we are going to look at taints and tolerations. We will taint a node. We will ensure that no pods can be scheduled in the tainted node. And finally, we will add a toleration to a pod to allow it from getting scheduled in the tainted node. We are going to start by adding an alias for kubectl and also the shorthand variable to quickly generate boiler template YAMLs. This is the same that we have been doing throughout all the demos and will help you gain some precious seconds during the exam. We are going to start by switching to the correct context, in this case, the one in module4. And we are going to taint the second node with the kubectl taint node, the name of the node, and then the taint itself. In this case, the key is dedicated, the value is special, and the action or the effect is NoSchedule. This effect means that no pod will be scheduled on the node. We also have another two effects PreferNoSchedule, and this means that Kubernetes will try to avoid scheduling pods in that node. And the last effect that we have is NoExecute, and this means that existing pods without a toleration are evicted from the node. We will execute the kubectl describe node and the name of the node to get more details about the node, but we will grep by taints just to make sure that the taint was successfully added, and here it is. Then, we will create a deployment with multiple replicas, 10 replicas in this case. This is to make sure that no pods are scheduled in that tainted node. After creating the deployment, we will execute the kubectl get pods -o wide command, and the -o wide command is to get additional details about the pods, including where it's running. If we expand the terminal, we see that no pods have been scheduled in node number 2. All the pods have been scheduled in the minikube node and also

in the node 3. Let's now create a pod definition. We are going to call the pod special-pod, and it's going to be running the Nginx image. Now, in that pod definition, we will add a toleration. We will open the pod definition, and we will add the toleration with key dedicated operator Equal, the value will be special, and the effect will be NoSchedule. So essentially, we are adding a toleration for the taint that we added in node 2. Let's go back and now create the pod with the kubectl apply -f command. And let's verify where the pod is running. We can see that the special pod has been scheduled in node 2 because it has the toleration. This will be all for this demo, so let's perform some cleanup. First, let's delete the pod with the kubectl delete pod command. Let's delete also the deployment with the kubectl delete deploy command, and finally, let's remove the taint with the kubectl taint node, the name of the node, and the name of the taint with a dash at the end to remove it.

Demo: Pod Anti-affinity

Pod anti-affinity is one of the most powerful ways to instruct the Kubernetes scheduler where and how to schedule your pods. In this demo, we will deploy three replicas of the same application and force them to spread across the three nodes to ensure high availability. We will do this with pod anti-affinity. As in other demos, we will start by setting up an alias for kubectl and also the shorthand variable to generate boiler template YAML without manually typing the flag. We will start the demo by switching to the module4-context. And then we will create a deployment definition with the kubectl deploy command. In this case, the deployment is going to use the Nginx image, and it's going to run three replicas. Now, we are going to add pod anti-affinity to the deployment definition. We will open the deployment, and we will add the affinity field. In this case, we are instructing the Kubernetes scheduler to not schedule a pod in the node if it already has a pod with the label app with value webserver. Let's now start the deployment with the kubectl apply -f command. If we now run the kubectl get pods -o wide, we will be able to see that all the pods are running in different nodes. So this means that if any node goes down, the application will still serve traffic from the other two pods, making the application highly available. This is just an example, but both affinity and anti-affinity is a very powerful mechanism. Apart from ensuring that no two instances of the same application are running on the same node, you can also configure applications to run co-located within the same node, for example, if your back-end application needs to connect to a database on the current node. Let's now do some clean up with the kubectl delete deploy command.

Pod Disruption Budgets

So far we have focused on the birth of a pod, getting it into the right hardware

using selectors and affinity. But as a certified Kubernetes administrator, you will spend just as much time on maintenance. Imagine that you have two worker nodes, and you need to upgrade the kernel on one of them. You will use a command called kubectl drain. This command tells all the pods on that node to pack their bags and move elsewhere. But there is a risk. What if your deployment only has one replica left because of a separate issue and you drain the node that it's sitting on? You have just caused an outage. This is where Pod Disruption Budget, or PDB, comes in. When working with your Kubernetes clusters, you can face two types of disruptions. Involuntary, like hardware failures, the cursed blue screen of death, or simply a network failure preventing two healthy nodes from communicating. These are unplanned disruptions. You don't have control over them. But you will also face planned or voluntary disruptions like cluster upgrades, node draining for maintenance, or scaling down a node pool to save costs. PDBs protect us from voluntary disruptions. A PDB is essentially a contract between the app owner and the cluster admin. It says you can perform your maintenance, but you must respect my safety floor. We define this floor using minAvailable or maxUnavailable. For example, if I set minAvailable to 2 on a 3 pods deployment, Kubernetes will allow me to drain one node. But if I try to drain a second node that will kill another pod, the API server will block my command until a new pod is safely running elsewhere.

Demo: Protect Pods with PDBs

Pod Disruption Budgets protect us against human failures during maintenance. In this demo, we will create a Pod Disruption Budget to require at least three replicas of our application. Then we will try to drain one of the nodes, and we are going to be unable to because the budget is not going to be met. To solve this, we will increase the number of replicas to meet the PDB. As in other demos, we will start by setting up an alias for kubectl. We will also export the shorthand variable to quickly generate boiler template YAMLs without manually typing the flags. Then, we will start by switching to the module4-context. After that, we will create a deployment called pdb-app that runs the Nginx image and has three replicas. We will execute the kubectl get pods -o wide command to see where the pods are running, and we can see that we have a pod running in each of the nodes. Let's now create a pdb called nginx-protection that uses the app=pdb-app as label selector and requires a minimum of three available replicas. Let's run the command. And we can see that the PDB has been created. Now we will try to bring node number 2, where we have one of the pods running, and we get an error because Kubernetes cannot evict the pod as it would violate the Pod Disruption Budget. Let's cancel this and increase the number of replicas to 4. This way, the new pod is going to be scheduled somewhere else, and whenever we try to drain node 2, there will be another three replicas available that will allow Kubernetes to meet the Pod Disruption Budget. Let's now execute the kubectl get pod command again just to see where the new container got the

schedule. And we can see that it's got a schedule in node number 3. Now, let's try to drain again node number 2. This time, the pod got evicted and also the node got drained. Let's now make sure that the evicted pod from node 2 is actually scheduled on another node. And we can see that it's got the schedule in the main node. Let's now perform some cleanup. First, let's delete the deployment. After that, we will delete the PDB. And finally, we will uncordon the node 2 to allow new pods to be scheduled on it.

Workload Autoscaling

Autoscaling Workloads

In production at 3 a.m. in the morning, if you get a sudden spike of users from the other side of the world who start using your application, who is going to scale up your deployment? Are you willing to sacrifice your sleep? I've got good news for you. Kubernetes once again can help you in this situation. Kubernetes has three main controllers that help automating the workload auto-scaling process. The first one is HPA, Horizontal Pod Autoscaler. It is a built-in controller that runs as part of the kube-controller-manager. It adjusts the number of replicas on a deployment or a StatefulSet based on metrics like CPU or memory. The next auto-scaling controller is the VPA, or Vertical Pod Autoscaler. This controller, instead of adjusting the number of replicas in your deployment or StatefulSet, it increases the number of CPU or memory requests so your pods can keep up with increased traffic. This is not a built-in controller and must be installed separately. And the last controller is the CA, or Cluster Autoscaler. It's also not a built-in controller and must be installed separately. This controller interacts with your cluster provider like GCP, AWS, or AKS to add more worker nodes when pods cannot be scheduled in the existing ones, and it can also remove nodes when they are underutilized. The CKA exam focuses on the standard Kubernetes primitives included in the core distribution, so you won't get tasked regarding the Vertical Pod Autoscaler or the Cluster Autoscaler. So let's focus on HPA. The HPA is a control loop. It periodically queries the metric servers to see how much CPU or memory your pods are using. It compares the actual usage against a target you have defined. If the usage is too high, it tells the deployment or the StatefulSet to add more replicas. If it's too low, it scales them back down to save costs. The HPA uses a simple ratio. If your target is 50% CPU and your pods are hitting 100%, Kubernetes will double your pod counts to bring that average back down to your target. When using the Horizontal Pod Autoscaler, a scaling decision happens continuously based on metrics like CPU or memory, but without protection, this can cause flapping. But what is flapping? Flapping happens when traffic slightly increases and decreases repeatedly in a short period of time. For example, you get a load increase and the HPA scales both up, quickly after the load decreases and the HPA scales down

the replicas. Then loads increase again and the number of replicas go up again. This sudden change of replica counts is very inefficient, but Kubernetes has a stabilization window to avoid flapping. When traffic drops, HPA does not immediately scale down. Instead, it waits for a period of time before reducing the replicas. By default, the stabilization window is 5 minutes, but it can be customized based on your needs.

Demo: Configuring HPA

In this demo, we are going to look at configuring HPA. We will first create a deployment. We will enable HPA for that deployment. Then, we will simulate a large number of traffic hitting the application, and we will monitor how the Horizontal Pod Autoscaler scales up the number of replicas in the deployment. As we have done in other demos, the first thing that we are going to do is set up an alias for kubectl and also export a shorthand variable to quickly generate boiler template YAMLs without manually typing the flags. Then, we will switch to the module5-context, same as you will do in the exam. And then we are going to create a deployment definition. The image that we are going to be using is hpa-example. This image is designed specifically for HPA tutorials. We will then edit the deployment to add the resources field. This is required for HPA to work as expected. We will go back, and then we will start the deployment by using the hpa-example -f apply command. We will also create a service for the deployment using the kubectl expose deploy, the name of the deployment, and the port 80. The service has been created, and now we are going to execute get pods to verify that the new pod is running. Now we are going to create the HPA. We will use the kubectl autoscale deployment, the name of the deployment, and then we will specify 50% of CPU. The minimum number of replicas, we will set it to 1, and the maximum number of replicas, it will be set to 10. Then, we will check the status of the HPA, and we can see that the HPA has been created. And in the TARGETS column, we see that the CPU is unknown/50%. So 50% is our target and unknown is the current usage. As we have just created, the HPA is still trying to identify what is the percentage of CPU that is used at the moment. So let's give it a second and then try to execute the command again. And here it is, 0% of the CPU is currently in use. Now, on a new terminal, we will generate load by using the kubectl run command. We will run a busybox image which connects to our deployment every 0.01 seconds, so very frequently. Let's run the command. And we will see that for each of the requests, it's printing OK, so our web server is working as expected. Now, let's go back to the other terminal and monitor the HPA status. We can see that the number of replicas has already scaled to three, and the current CPU usage is 15. Let's give it a few more minutes and we will see the number of replicas going up even more. We can see that the current CPU usage is 109% and the replica counts has jumped to 6. Now it has jumped to 7, and it will continue to do so until it reaches 10 replicas. That is the maximum replica counts that we have defined. Let's now perform some

cleanup. The first thing that we are going to do is kill the load generated bug. Then we will delete the Horizontal Pod Autoscaler with the kubectl delete hpw, and finally, we will delete the service with the kubectl delete svc and the name of the service.

Course Summary and Exam Tips

In this course, we have covered a lot of ground. We talked about the different types of workload controllers available in Kubernetes and when to use them, deployments, DaemonSets, CronJobs. We learned about the different rollout strategies and also cover how to do rollbacks when things go wrong. We learned that code and config should never live together. By using ConfigMaps and secrets, you can move your applications across environments without changing a single line of the image. We mastered how to use a resource request to get past admissions and how to use taints, toleration, and affinity to navigate the scheduler's logic. And finally, we saw how to make the cluster elastic using HPA to scale our pod automatically based on real-time demand. To wrap up, here are my top tips for the exam. Set up an alias for kubectl and also for –dry-run=clients output YAML. This will save you a massive amount of time and helps avoid typos. Learn and use the short name of the resource types. For example, use cm for configmap. You want to spend the least time possible typing. Always check your context. The exam will have multiple clusters and namespaces. Before starting any task, make sure that you are working on the current cluster and in the current namespace by using the kubectl config use-context command given in the prompt. Use the official documentation during your exam, especially the quick reference page. Don't be afraid to use it to look up the exact syntax for a resource you are not 100% sure about. Remember that only one additional tab is allowed. Spend some time playing with the Kubernetes docs page ahead of the exam. Knowing how to quickly navigate and search this documentation is crucial for success. And finally, manage your time efficiently. If you are stuck on a question, flag it and move on. It's better to complete five easy tasks than to get stuck on a difficult one. At the end of the day, you don't need to solve all the tasks to pass the exam, just 66% of them. And this brings us to the end of the course. You are now equipped with the core knowledge required for the workload and scheduling portion of the CKE exam. Use the demo files included in the Resource tab to practice more. Thank you for joining me, and good luck on your exam.